NAME : PRIYASUDHAN P **REG N0 :** 1912260182

COURSE : PYTHON PROGRAMMING **CODE :** CSA0820

SLOT : D **FACULTY :** KEERTHANA S

GITHUB LINK : https://github.com/priyasudhanp87-creator/python-programming-assignment.git

TASK 1 REPORT

1. Aim and Objective

Create a Python program that records milk delivery details for a ten-day period and calculates the payment for each record. The program accepts farmer ID, day number, litres, fat %, SNF %, shift and a manually entered rate per litre. Payment = litres x rate. The rate is entered from the applicable official rate chart; sample rates here are illustrative only and are not claimed to be current Aavin rates. No fat/SNF rate formula, deductions or incentives are calculated.

Input	Purpose / validation
Number of entries	Whole number, at least 1
Farmer ID	Non-empty text
Day number	Integer 1 to 10
Milk litres	Number >= 0
Fat percentage	3.0 to 7.0 (current code)
SNF percentage	Number >= 7.5
Shift	Morning or Evening
Rate per litre	Positive number, entered manually

2. Algorithm / Approach

1. Read and validate the number of entries.
2. For each entry read and validate farmer ID, day, litres, fat, SNF, shift and rate.
3. Store each entry as a MilkEntry object.
4. Compute payment = litres x rate for every record and accumulate the total.
5. Display the summary table, total and limitations.

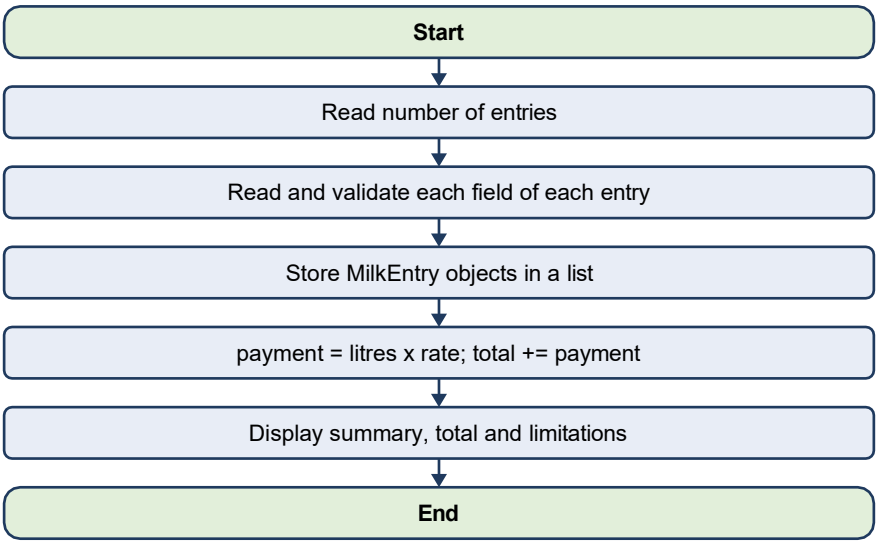
3. Pseudocode

```
START
  Display title and rate-chart
  instruction Read number of entries
  Repeat for each entry:
    Read and validate farmer ID, day, litres, fat, SNF, shift,
    rate Create a MilkEntry record
  total = 0
  For each record:
```

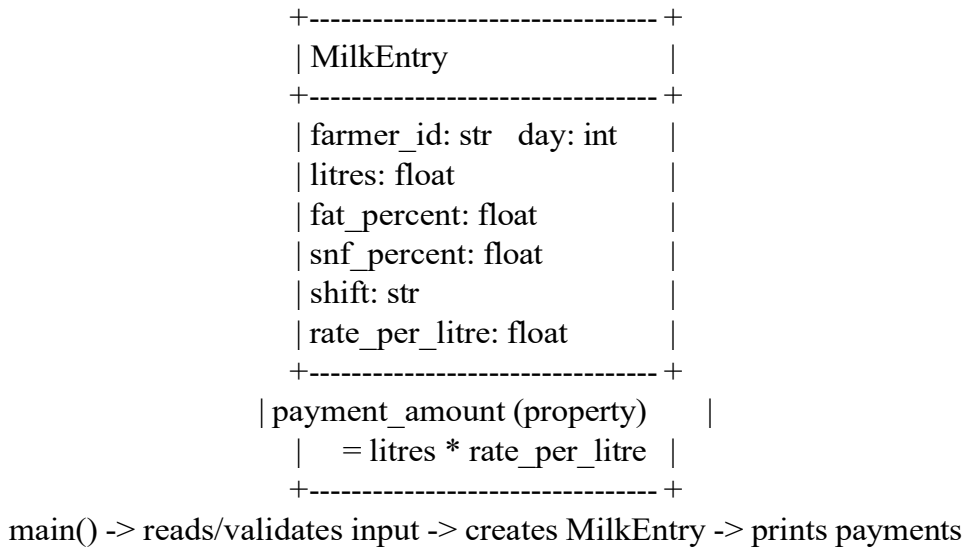
payment = litres * rate_per_litre
Display record and payment
total = total + payment
Display total and
limitations
END

4. UML Diagram

Activity / program flow:



Class / structure diagram:



5. Source Code

```
"""
Task 1: Aavin Milk Payment Record
The rate per litre must be entered manually from the applicable official
rate chart. """
from dataclasses import
```

```
dataclass import math

MIN_FAT_PERCENT = 3.0
MAX_FAT_PERCENT = 7.0
MIN_SNF_PERCENT = 7.5
FIRST_DAY = 1
LAST_DAY = 10


@dataclass
class
    MilkEntry:
    farmer_id:
    str day: int
    litres: float
    fat_percent: float
    snf_percent: float
    shift: str
    rate_per_litre:
    float

    @property
    def payment_amount(self):
        return round(self.litres * self.rate_per_litre, 2)


def
    read_text(prom
    pt): while True:
        value =
        input(prompt).strip() if
        value:
            return value
        print("Error: This field cannot be empty.")


def read_number(prompt, minimum, maximum=None,
    strict_min=False): while True:
        try:
            value = float(input(prompt).strip())
            min_ok = value > minimum if strict_min else value >=
            minimum max_ok = maximum is None or value <=
            maximum
            if math.isfinite(value) and min_ok and
                max_ok: return value
        except
            ValueError:
            pass
        if maximum is None:
```

```
            print("Error: Enter a valid number within the allowed
        range.") else:
            print(f"Error: Enter a number from {minimum} to {maximum}.")


def read_integer(prompt, minimum,
    maximum=None): while True:
        try:
            value = int(input(prompt).strip())
            if value >= minimum and (maximum is None or value
                <= maximum): return value
        except
            ValueError:
            pass
        print("Error: Enter a valid whole number.")


def read_shift():
    while True:
        shift = read_text("Shift (Morning/Evening):
        ").lower() if shift == "morning":
            return
        "Morning" if shift
        == "evening":
            return "Evening"
        print("Error: Enter Morning or Evening.")


def main():
    print("=== Aavin Milk Payment Record ===")
    print("Enter the applicable rate from your current official Aavin rate
    chart.") count = read_integer("Number of entries for the 10-day
    period: ", 1)
    entries =[]
    for i in range(1, count + 1):
        print(f"\nEntry {i}")
        farmer_id = read_text("Farmer ID: ")
        day = read_integer("Day number (1-10): ", FIRST_DAY,
        LAST_DAY) litres = read_number("Milk litres (>= 0): ",
        0)
        fat = read_number("Fat percentage (3.0-7.0): ",
                          MIN_FAT_PERCENT, MAX_FAT_PERCENT)
        snf = read_number("SNF percentage (>= 7.5): ",
        MIN_SNF_PERCENT) shift = read_shift()
        rate = read_number(
            "Rate per litre in rupees (manual official chart
            entry): ", 0, strict_min=True)
        entries.append(MilkEntry(farmer_id, day, litres, fat, snf, shift, rate))
```

```
    print("\n=== Payment Summary ===")
    print("Farmer ID | Day | Litres | Fat% | SNF% | Shift | Rate/Litre |
    Amount") total = 0.0
    for entry in entries:
        amount =
        entry.payment_amount
        total += amount
        print(f"{entry.farmer_id} | {entry.day} | {entry.litres:.2f} | "
              f"{entry.fat_percent:.2f} | {entry.snf_percent:.2f} | "
              f"{entry.shift} | Rs {entry.rate_per_litre:.2f} | Rs
              {amount:.2f}")
    print(f"Ten-day total for entered records: Rs {total:.2f}")
    print("Note: Deductions, incentives, and adjustments are not
    included.") print("Confirm rates and quality rules with your local
    Aavin society.")


if _name_== "_main_
    ": main()
```

6. Sample Input

```
2
F00
1 1
10
4.3
8.2
Morni
ng 44
F00
2 2
12.5
4.5
8.4
Eveni
ng 45
```

7. Expected Output

```
=== Payment Summary ===
Farmer ID | Day | Litres | Fat% | SNF% | Shift | Rate/Litre |
Amount F001 | 1 | 10.00 | 4.30 | 8.20 | Morning | Rs 44.00 |
Rs 440.00
F002 | 2 | 12.50 | 4.50 | 8.40 | Evening | Rs 45.00 | Rs
562.50 Ten-day total for entered records: Rs 1002.50
Note: Deductions, incentives, and adjustments are not
included. Confirm rates and quality rules with your
local Aavin society.
```

8. Test Table

ID	Input / scenario	Expected result
TC-01	10 L, rate Rs 44	Payment = Rs 440.00
TC-02	12.5 L, rate Rs 45	Payment = Rs 562.50
TC-03	Day 0 or 11	Rejected; prompt repeats
TC-04	Blank farmer ID	Rejected; prompt repeats
TC-05	Shift not Morning/Evening	Rejected; prompt repeats
TC-06	Fat < 3.0 or > 7.0	Rejected (current validation)
TC-07	SNF < 7.5	Rejected; prompt repeats
TC-08	Rate 0 or negative	Rejected; rate must be positive

9. Calculation check

F001: 10.00 L x Rs 44.00 = Rs 440.00. F002: 12.50 L x Rs 45.00 = Rs 562.50. Total = Rs 1,002.50.

10. Complexity Analysis

Time O(n): each of n entries is read once and traversed once more for the summary. Space O(n): all MilkEntry records are stored before printing.

11. Limitations and Notes

- Sample rates are for demonstration only; verify the rate chart with the relevant society.
- The code accepts fat 3.0% to 7.0%. If the assignment specifies 2.0% to 7.0%, change MIN_FAT_PERCENT and the prompt text.
- Output shown is an expected transcript, not a screenshot of an actual terminal session.

12. Conclusion

The program records ten-day milk deliveries with validated input and computes payments from a manually entered official rate, without inventing a rate formula.

TASK 2 REPORT

Explore Values and Types in the Python Interpreter | 4 Marks

1. Aim and Objective

Use the Python interpreter and a runnable program to analyse built-in types, the division operators, floating-point precision and safe ways to total money. Complete total_decimal(items) with Decimal and compare it with float, Decimal(float(amount)), integer paise and round-after-each-addition.

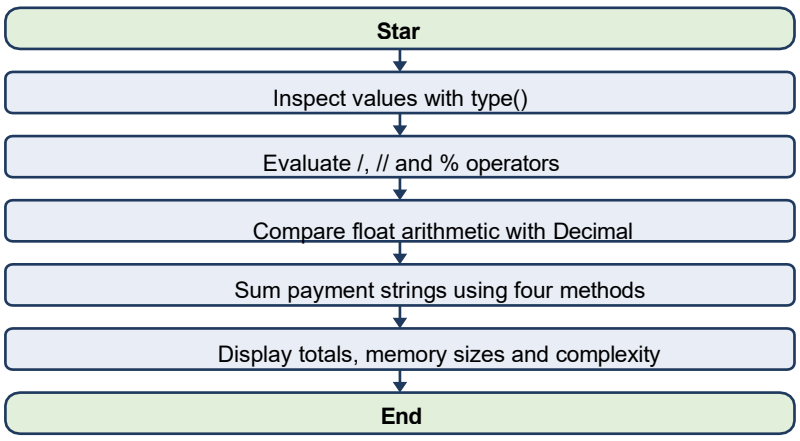
2. Algorithm / Approach

1. Display type() of 10, 10.0, "10", True and [10, 8.5].
2. Evaluate /, // and % including negative floor division.
3. Compare 0.1 + 0.2 with 0.3, then repeat with Decimal strings.
4. Define total functions using float, Decimal strings, integer paise and per-step rounding.
5. Run the experiment: 1,000 payments of Rs 33.40 plus three adjustments of Rs 0.10; compare totals and object sizes.
6. Report time and auxiliary-space complexity.

3. Pseudocode

```
START
  Display sample values and their
  type() Display results of /, // and %
  Display float and Decimal comparisons
  Define total_float, total_decimal,
         total_decimal_from_float, total_integer_paise,
         total_rounded_float
  payments = ["33.40"] * 1000 + ["0.10"] * 3
  Calculate and display each method's
  total Display memory sizes and
  complexity
END
```

4. UML Diagram



5. Source Code

```
from decimal import
Decimal import sys

def
    total_float(items
    ): total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):        # TODO 1
    (completed) total = Decimal("0.00")
    for amount in items:
        total +=
    Decimal(amount) return
    total

def
    total_decimal_from_float(ite
    ms): total = Decimal("0.00")
    for amount in items:
        total +=
    Decimal(float(amount)) return
    total

def
    total_integer_paise(ite
    ms): paise = 0
    for amount in items:
        paise += round(float(amount)
    * 100) return paise

def
    total_rounded_float(ite
    ms): total = 0.0
    for amount in items:
        total = round(total + float(amount),
    2) return total

payments = ["33.40"] * 1000 + ["0.10"] * 3
print("Float total:", total_float(payments))
print("Decimal total:",
total_decimal(payments))
print("Decimal(float) total:",
total_decimal_from_float(payments)) print("Integer
paise:", total_integer_paise(payments)) print("Rounded
```

```
float total:", total_rounded_float(payments))
print(sys.getsizeof(1.0),  sys.getsizeof(Decimal("1.00")),  sys.getsizeof(3340030))
```

6. Expected Output

```
10 -> <class 'int'>         10.0 -> <class
'float'> '10' -> <class 'str'>True -> <class
'bool'> [10, 8.5] -> <class 'list'>
7 / 2 = 3.5    7 // 2 = 3    7 % 2 = 1
-7 // 2 = -4   -7 % 2 = 1
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3 -> False
Decimal("0.1") + Decimal("0.2") == Decimal("0.3") -> True

Payment count: 1003
Float total: 33400.30000000061
Decimal total: 33400.30
Decimal(float(amount)) total: 33400.2999999999998578931182 78
Integer paise: 3340030      (Rs
33400.30) Rounded float total:
33400.3
Memory (bytes, platform-dependent): float 24, Decimal 104, int 28
```

Totals were verified by running the functions. Object sizes vary by Python build.

7. Test Table

ID	Test	Expected result	Status
TC-01	Types of 10, 10.0, "10", True, [10, 8.5]	int, float, str, bool, list	Pass
TC-02	7/2, 7//2, 7%2	3.5, 3, 1	Pass
TC-03	-7//2 and -7%2	-4 and 1	Pass
TC-04	0.1+0.2 == 0.3	False	Pass
TC-05	Decimal string equality	True	Pass
TC-06	1,000 x 33.40 + 3 x 0.10	Decimal total Rs 33,400.30	Pass

Key observations

Floor division rounds toward negative infinity, so -7 // 2 = -4. Binary floating point cannot represent many decimal fractions exactly, so 0.1 + 0.2 != 0.3. Decimal built from strings is exact for decimal input.

Decimal(float(amount)) imports the float's approximation and therefore does not preserve the original decimal text. Integer paise is exact for amounts with two decimal places. Rounding after each step hides the error but the arithmetic is still binary float.

8. Complexity Analysis

Time O(n) for each total function (each payment visited once). Auxiliary space O(1) for the running total; the payment list itself needs O(n).

9. Limitations and Notes

- Object sizes are platform-dependent.
- This report covers the fixed experiment only; add runtime-input testing if your source includes it.

10. Conclusion

For money entered as decimal text, use Decimal(string) or integer paise. Avoid float accumulation and Decimal(float(amount)).

TASK 3 REPORT

1. Aim and Objective

Define limits as named constants so revisions are made in one place. Demonstrate naming conventions, Python's constant convention, dynamic typing, multiple assignment, swapping and runtime input.

2. Algorithm / Approach

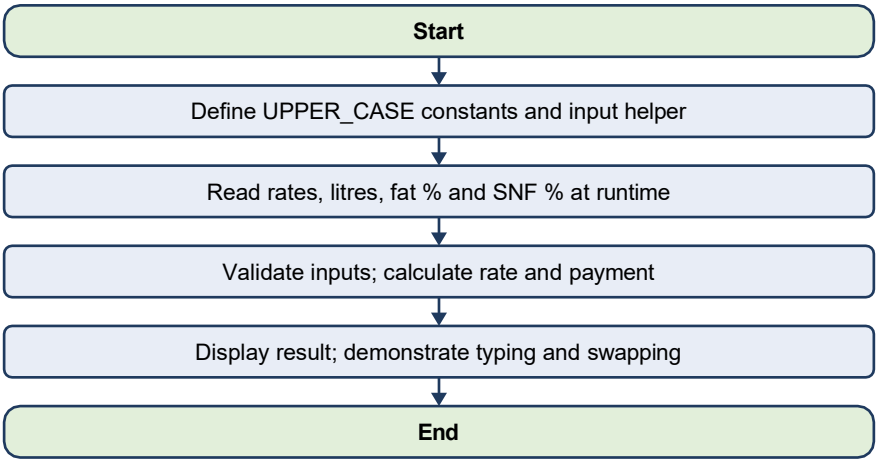
1. Declare limits with UPPER_CASE names.
2. Read rate values and milk details at runtime.
3. Validate numeric values and fat/SNF limits.
4. Calculate an illustrative rate and payment.
5. Demonstrate int-to-str dynamic typing and swapping.
6. Compare magic numbers, named constants and a RATE_CARD dictionary.

3. Pseudocode

```
START
  Define MIN_FAT_PERCENT, MAX_FAT_PERCENT, MIN_SNF_PERCENT
  Read base rate, fat rate, SNF rate, litres, fat %, SNF %
  Validate numeric inputs and allowed ranges
  rate = base + fat% * fat_rate + snf% *
  snf_rate payment = round(litres * rate, 2)
  Display rate and payment
  value = 10, then value = "ten"; display
  types a, b = 5, 9; swap using a, b = b, a
END
```

4. UML Diagram

Activity / program flow:



5. Source Code

```
# TASK 3: Choose Variables and Name the Constants
# Illustrative formula only; not an official Aavin rate
formula. MIN_FAT_PERCENT = 3.0
MAX_FAT_PERCENT = 7.0
MIN_SNF_PERCENT = 7.5


def read_float(prompt, minimum=None,
    maximum=None): while True:
        try:
            value = float(input(prompt))
            if minimum is not None and value <
                minimum: print(f"Enter a value
                >= {minimum}.") continue
            if maximum is not None and value >
                maximum: print(f"Enter a value
                <= {maximum}.") continue
            return value
        except
        ValueError:
            print("Invalid input. Enter a number.")


print("=== Named Constants and Runtime Input
===") BASE_RATE = read_float("Enter base rate
per litre: ", 0) FAT_RATE = read_float("Enter fat
rate: ", 0)
SNF_RATE = read_float("Enter SNF rate: ", 0)
litres = read_float("Enter milk quantity in litres: ", 0)
fat_percent = read_float("Enter fat percentage (3 to 7): ",
                         MIN_FAT_PERCENT, MAX_FAT_PERCENT)
snf_percent = read_float("Enter SNF percentage
                         (minimum 7.5): ",
                         MIN_SNF_PERCENT)

rate_per_litre = BASE_RATE + fat_percent * FAT_RATE + snf_percent
* SNF_RATE payment_amount = round(litres * rate_per_litre, 2)

print("\n=== Payment Result ===")
print(f"Base rate: Rs
{BASE_RATE:.2f}")
print(f"Rate per litre: Rs
{rate_per_litre:.2f}") print(f"Milk quantity:
{litres:.2f} litres") print(f"Payment
amount: Rs {payment_amount:.2f}")

print("\n=== Dynamic
```

```
Typing ===") value = 10
print(value, type(value)._
name_) value = "ten"
print(value, type(value)._name_)

print("\n=== Swapping
===") a, b = 5, 9
print("Before swap:", a,
b) a, b = b, a
print("After swap:", a, b)
print("\nUPPER_CASE is a convention; Python does not enforce constants.")
```

6. Sample Input

```
Enter base rate per litre:
30 Enter fat rate: 0.40
Enter SNF rate: 0.30
Enter milk quantity in litres: 10
Enter fat percentage (3 to 7): 4.3
Enter SNF percentage (minimum 7.5): 8.2
```

7. Expected Output

```
=== Payment Result
=== Base rate: Rs
30.00
Rate per litre: Rs 34.18
Milk quantity: 10.00
litres Payment amount:
Rs 341.80

=== Dynamic Typing ===
10 int
ten str

=== Swapping
=== Before
swap: 5 9
After swap: 9 5
```

Correction: rate = 30 + 4.3 x 0.40 + 8.2 x 0.30 = 34.18 (the earlier draft showed 34.22 / 342.20, which was an arithmetic error).

8. Test Table

ID	Input / test	Expected result	Status
TC-01	Valid numeric values	Rate 34.18; payment 341.80	Pass
TC-02	Non-numeric input	Reprompt with error	Pass
TC-03	Fat outside 3 to 7	Validation and reprompt	Pass
TC-04	int then str	Types int then str	Pass
TC-05	Swap 5 and 9	9 and 5	Pass

9. Naming rules and dynamic typing

Identifiers may contain letters, digits and underscores, cannot start with a digit and are case-sensitive. Use snake_case for variables and UPPER_CASE for constants; Python does not enforce constants. A name can refer first to an int and later to a str. Multiple assignment lets a, b = b, a swap values without a temporary variable.

10. Complexity Analysis

If a rate is repeated as a magic number in k expressions, revision needs k edits (O(k) maintenance); a named constant needs one edit (O(1)). A RATE_CARD dictionary groups rates by key but uses extra memory.

Calculation time stays O(1) for all approaches; constants improve maintainability, not speed.

11. Limitations and Notes

- The rate formula is illustrative, not an official Aavin calculation. Replace it if the assignment gives a specific formula.

12. Conclusion

The program demonstrates named limits, runtime input, dynamic typing and swapping, with the maintainability benefit of named constants.

TASK 4 REPORT

Write Correct Expressions | 4 Marks

1. Aim and Objective

Compute fat/SNF step counts, rate per litre and amount; analyse operator precedence, floating-point precision, round() versus int()/math.floor(), chained comparisons and Boolean operators.

2. Algorithm / Approach

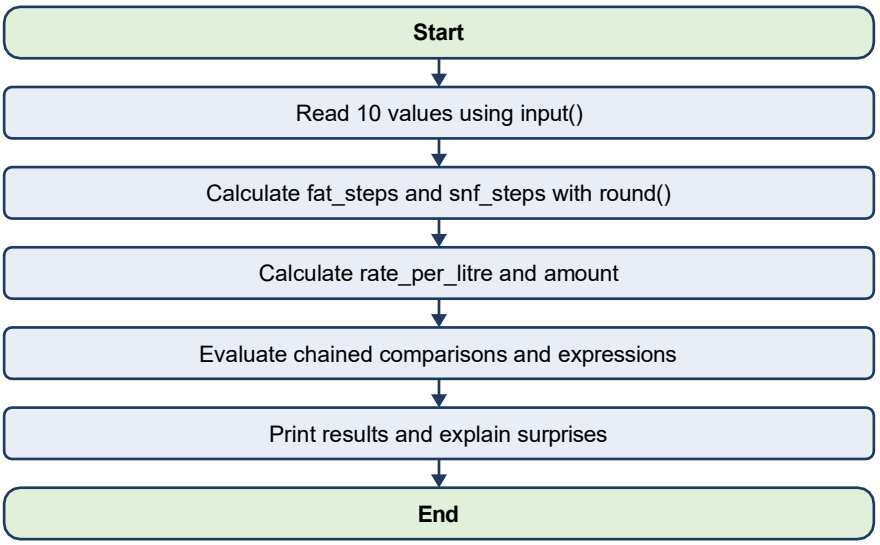
1. Read all values at runtime.
2. Calculate step counts with round().
3. Calculate rate per litre and amount.
4. Validate inputs with chained comparisons and Boolean operators.
5. Print unexpected-expression examples.

3. Pseudocode

```
START
  Read all values at runtime
  fat_steps = round((fat - base_fat) /
  fat_step_size) snf_steps = round((snf -
  base_snf) / snf_step_size)
  rate = base_rate + fat_steps*fat_step_rate +
  snf_steps*snf_step_rate amount = round(litres * rate, 2)
  fat_valid = 2.0 <= fat <=
  7.0 snf_valid = snf >=
  base_snf
  valid = fat_valid and snf_valid and litres
  >= 0 Print results and expression
  demonstrations
END
```

4. UML Diagram

Activity / program flow:



5. Source Code

```
# TASK 4: Write Correct
Expressions import math

fat = float(input("Enter fat percentage (e.g. 4.2):
")) snf = float(input("Enter SNF percentage (e.g.
8.2): ")) litres = float(input("Enter milk quantity
in litres: ")) base_rate = float(input("Enter base
rate per litre: ")) fat_step_rate = float(input("Enter
rate per fat step: ")) snf_step_rate =
float(input("Enter rate per SNF step: "))
fat_step_size = float(input("Enter fat step size (e.g. 0.1): "))
snf_step_size = float(input("Enter SNF step size (e.g. 0.1): "))
base_fat = float(input("Enter base fat percentage (e.g. 3.5): "))
base_snf = float(input("Enter base SNF percentage (e.g. 7.5):
"))

# round() removes binary-float noise from the step
counts fat_steps = round((fat - base_fat) /
fat_step_size) snf_steps = round((snf - base_snf) /
snf_step_size)
rate_per_litre = base_rate + fat_steps * fat_step_rate + snf_steps *
snf_step_rate amount = round(litres * rate_per_litre, 2)

print("\n--- Calculation Result ---
") print("Fat steps:", fat_steps)
print("SNF steps:", snf_steps)
print(f"Rate per litre: Rs {rate_per_litre:.2f}")
print(f"Amount: Rs {amount:.2f}")

fat_valid = 2.0 <= fat <= 7.0
snf_valid = snf >= base_snf
valid = fat_valid and snf_valid and litres >=
0 print("Fat valid:", fat_valid)
print("SNF valid:", snf_valid)
print("Input valid:", valid)

raw_steps = (4.2 - 3.5) / 0.1
print("\n--- Floating-point / rounding demonstrations ---")
print("(4.2 - 3.5) / 0.1 =", repr(raw_steps))
print("round(raw_steps) =", round(raw_steps))
print("int(raw_steps) =", int(raw_steps))
print("math.floor(raw_steps) =", math.floor(raw_steps))

print("\n--- Three unexpected expressions --
-") print("0.1 + 0.2 =", repr(0.1 + 0.2))
print("0.1 + 0.2 == 0.3 ->", 0.1 + 0.2 == 0.3)
print("-7 // 2 =", -7 // 2, "; int(-7 / 2) =", int(-7 / 2), ";
```

```
        floor(-7 / 2) =", math.floor(-7 / 2))
print("2 ** 3 ** 2 =", 2 ** 3 ** 2, "(** is right-associative)")
print("-2 ** 2 =", -2 ** 2, "but (-2) ** 2 =", (-2) ** 2)
```

6. Sample Input

```
4.2
8.2
10
30
0.40
0.30
0.1
0.1
3.5
7.5
```

7. Expected Output

```
--- Calculation Result -
--Fat steps: 7
SNF steps: 7
Rate per litre: Rs 34.90
Amount: Rs 349.00
Fat valid: True
SNF valid:
True Input
valid: True

--- Floating-point / rounding
demonstrations ---(4.2 - 3.5) / 0.1 =
7.000000000000002
round(raw_steps) = 7
int(raw_steps) = 7
math.floor(raw_steps) = 7

--- Three unexpected expressions ---
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3 -> False
-7 // 2 = -4 ; int(-7 / 2) = -3 ; floor(-7 / 2) = -4
2 ** 3 ** 2 = 512 (** is right-associative)
-2 ** 2 = -4 but (-2) ** 2 = 4
```

8. Test Table

ID	Test	Result	Status
TC-01	Fat steps	7	Pass
TC-02	SNF steps	7	Pass
TC-03	0.1+0.2==0.3	False	Pass
TC-04	Negative division	int = -3; floor = -4	Pass
TC-05	Exponentiation	512; -4; 4	Pass

9. Expression analysis

** binds tighter than unary minus, so -2 ** 2 = -4 while (-2) ** 2 = 4. ** is right-associative, so 2 ** 3 ** 2 = 512.

Binary float approximation makes (4.2 - 3.5) / 0.1 evaluate to 7.000000000000002; round() gives 7. int() truncates toward zero while math.floor() rounds down, so int(-7 / 2) = -3 and math.floor(-7 / 2) = -4. The chained comparison 2.0 <= fat <= 7.0 equals 2.0 <= fat and fat <= 7.0.

10. Complexity Analysis

For fixed inputs each calculation is O(1) time and O(1) auxiliary space; round(), int() and math.floor() are constant time on scalars.

11. Limitations and Notes

- The rate formula is illustrative; replace it if the assignment specifies a different one.

12. Conclusion

The program uses runtime input to compute step counts, rate and payment and demonstrates precision, precedence, conversions and Boolean logic.

TASK 5 REPORT

Read Input and Convert Types | 3 Marks

1. Aim and Objective

Read clerk entries, convert numeric text, handle invalid input, calculate the amount and print aligned results.

2. Algorithm / Approach

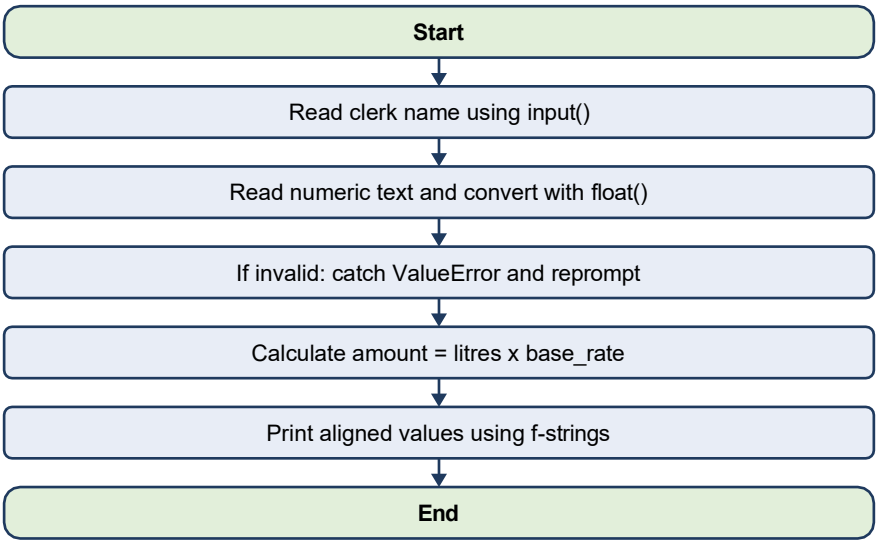
1. Read the clerk name with input().
2. Read numeric entries as strings.
3. Convert with float() inside a try/except ValueError loop.
4. Calculate amount = litres x base_rate.
5. Print aligned results using f-strings.

3. Pseudocode

```
START
  clerk_name = input()
  fat_percent =
  read_number(input) litres =
  read_number(input) base_rate
  = read_number(input) amount
  = litres * base_rate
  Print clerk name, fat %, litres, rate and amount in aligned
format END
```

4. UML Diagram

Activity / program flow:



5. Source Code

```
# TASK 5: Read Input and Convert Types

def
    read_number(pro
    mpt): while
    True:
        text =
        input(prompt).strip() try:
            return float(text)
        except ValueError:
            print("Invalid number! Enter a value like 8.5.")


print("=== Clerk Entry ===")
clerk_name = input("Enter clerk name: ").strip()
fat_percent = read_number("Enter fat percentage (example
8.5): ") litres = read_number("Enter milk quantity in litres:
")
base_rate = read_number("Enter base rate per litre

(Rs): ") amount = litres * base_rate

print("\n=== Entry Summary ===")
print(f"{'Clerk':<18}: {clerk_name}")
print(f"{'Fat percentage':<18}: {fat_percent:>8.2f}%")
print(f"{'Milk quantity':<18}: {litres:>8.2f} L")
print(f"{'Base rate':<18}: Rs {base_rate:>8.2f}")
print(f"{'Amount':<18}: Rs {amount:>8.2f}")
```

6. Sample Input

```
Enter clerk name: Arun
Enter fat percentage (example 8.5):
8.5 Enter milk quantity in litres: 10
Enter base rate per litre (Rs): 30
```

7. Expected Output

```
=== Entry Summary
=== Clerk        :
Arun
Fat percentage  :
                 8.50%
Milk quantity   :      10.00
L Base rate      : Rs  30.00
Amount           : Rs    300.00
```

Input prompts are omitted from the output shown.

8. Test Table

Input	Behaviour	Reason
8.5	Accepted as 8.5	Valid decimal text
8,5	Reprompt	Comma is not parsed by float()
eight	Reprompt	ValueError
12	Accepted as 12.0	Valid numeric text

9. Alternative approach

isdigit() handles some integer strings but does not validate decimals such as 8.5 or negative numbers. try/except around float(text) tests directly whether Python can parse a number, and is clearer for general numeric input.

10. Complexity Analysis

Both approaches take O(m) time for a string of length m and O(1) auxiliary space beyond the input text and the scalar.

11. Conclusion

input() returns str; float() converts valid numeric text and raises ValueError for invalid text. f-strings align values and show two decimal places.

TASK 6 REPORT

1. Aim and Objective

Divide the milk-payment calculation into reusable functions that use parameters and return values, are documented with docstrings, and separate local variables from global rate-card constants. The rate card is the one used in Task 7: base Rs 30.00 at 3.5% fat and 8.5% SNF; Rs 0.40 per 0.1% fat step; Rs 0.30 per 0.1% SNF step.

2. Algorithm / Approach

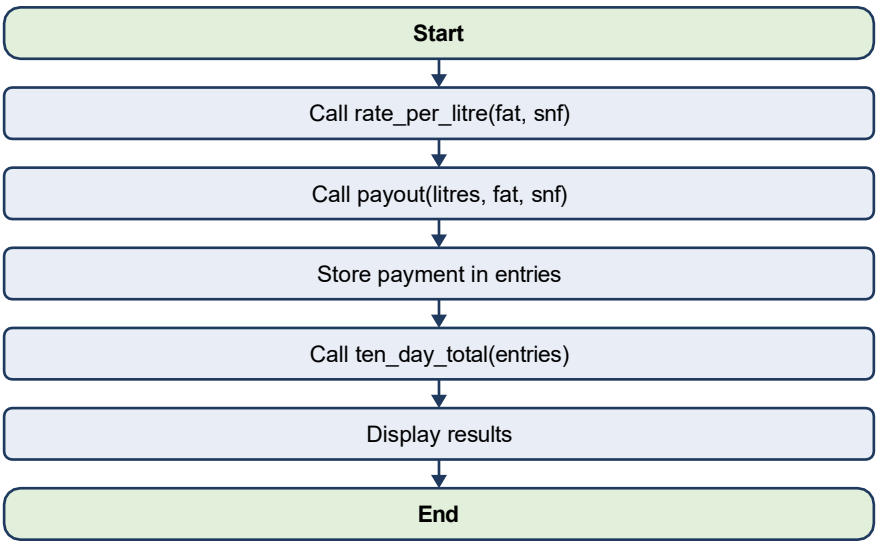
1. Define module-level constants for the rate card.
2. rate_per_litre(fat, snf) returns the rate.
3. payout(litres, fat, snf) returns the payment.
4. ten_day_total(entries) returns the sum of stored payments.

3. Pseudocode

```
FUNCTION rate_per_litre(fat,
  snf) fat_steps = round((fat -
  3.5) * 10) snf_steps =
  round((snf - 8.5) * 10)
  RETURN round(BASE + fat_steps*FAT_STEP +
snf_steps*SNF_STEP, 2) FUNCTION payout(litres, fat, snf)
  RETURN round(litres * rate_per_litre(fat,
snf), 2) FUNCTION ten_day_total(entries)
  total = 0
  FOR each entry: total = total +
  entry["payment"] RETURN round(total,
  2)
```

4. UML Diagram

Activity / program flow:



5. Source Code

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30


def rate_per_litre(fat, snf):
    """Calculate the milk rate per litre."""
    fat_steps = round((fat - 3.5) * 10)
    snf_steps = round((snf - 8.5) * 10)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps
    * SNF_STEP_VALUE return round(rate, 2)


def payout(litres, fat, snf):
    """Calculate the payment for a quantity of
    milk."""" return round(litres *
    rate_per_litre(fat, snf), 2)


def ten_day_total(entries):
    """Calculate the total of stored payment
    entries."""" total = 0.00
    for entry in entries:
        total +=
    entry["payment"] return
    round(total, 2)
```

6. Sample Input

```
payout(10, 3.5, 8.5)
payout(8.5, 4.2, 8.7)
payout(12, 3.1, 8.3)
```

7. Expected Output

Test	Litres	Fat%	SNF%	Rate (Rs/L)	Payment (Rs)
1	10	3.5	8.5	30.00	300.00
2	8.5	4.2	8.7	33.40	283.90
3	12	3.1	8.3	27.80	333.60

8. Test Table

Function	Parameters	Returns	Purpose
rate_per_litre	fat, snf	Rate per litre	Calculates rate from fat and SNF steps
payout	litres, fat, snf	Payment	Multiplies litres by the calculated rate
ten_day_total	entries	Total payment	Adds stored payment values

9. Local and global scope

BASE_RATE, FAT_STEP_VALUE and SNF_STEP_VALUE are module-level constants that functions read but never change. fat_steps, snf_steps, rate and total are local to their functions.

10. Alternative designs

A long script repeats logic and is harder to test. Functions reduce duplication and aid testing. Lambda expressions suit short one-line expressions but are less readable for multi-step logic.

11. Complexity Analysis

One rate or payment calculation is O(1); totalling n stored entries is O(n). Stored entries need O(n) space; a running total needs O(1) extra space.

TASK 7 REPORT

1. Aim and Objective

Place the milk rate card and calculation functions in a reusable module (milkrate.py) and import and call them from a separate script (test_milkrate.py).

2. Algorithm / Approach

1. Create milkrate.py with the rate-card constants and three functions.
2. Add an if __name__ == '__main__' demonstration block.
3. Create test_milkrate.py that uses three import styles.
4. Keep both files in the same folder (or on sys.path).

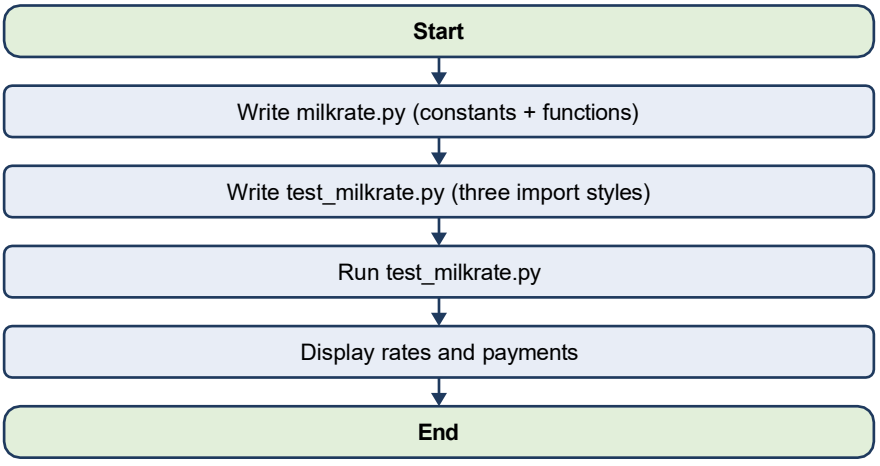
3. Pseudocode

milkrate.py
BASE_RATE = 30.00 ; FAT_STEP_VALUE = 0.40 ; SNF_STEP_VALUE = 0.30
fat_steps = round((fat - 3.5) * 10)
snf_steps = round((snf - 8.5) * 10)
rate = BASE_RATE + fat_steps*FAT_STEP_VALUE +
snf_steps*SNF_STEP_VALUE payment = litres * rate
(both rounded to 2 decimals)

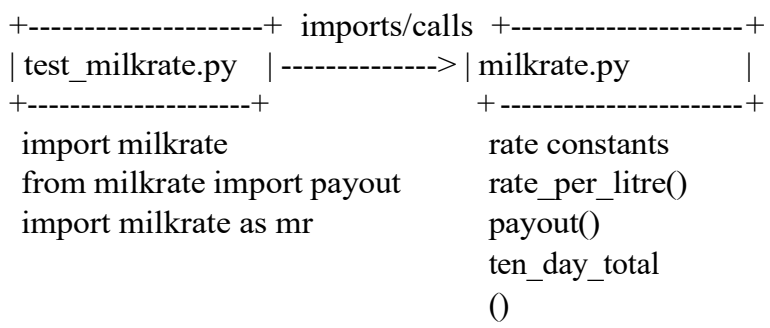
test_milkrate.py
import milkrate ; from milkrate import payout ; import
milkrate as mr call the functions and print results

4. UML Diagram

Activity / program flow:



Class / structure diagram:



5. Source Code

```
# ---------- File 1: milkrate.py ----------
"""Rate card of the village milk co-
operative."""  BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30


def rate_per_litre(fat, snf):
    """Return the rate per litre rounded to two
    decimals."""  fat_steps = round((fat - 3.5) * 10)
    snf_steps = round((snf - 8.5) * 10)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps
    * SNF_STEP_VALUE return round(rate, 2)


def payout(litres, fat, snf):
    """Return the payment rounded to two
    decimals."""  return round(litres *
    rate_per_litre(fat, snf), 2)


def ten_day_total(entries):
    """Return the sum of stored payment
    entries."""  total = 0.00
    for entry in entries:
        total +=
    entry["payment"] return
    round(total, 2)


if __name__ == "__main__":
    for litres, fat, snf in [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:5} L fat {fat} snf {snf} rate {rate:6.2f} pay {pay:8.2f}")


# ---------- File 2: test_milkrate.py ----------
"""Demonstrate importing and using the milkrate
module."""  import milkrate
from milkrate import
payout import milkrate
as mr


def main():
    """Test three module import
```

```
    styles."""
    rate1 = milkrate.rate_per_litre(3.5, 8.5)
    print(f"Method 1 - Rate: {rate1:.2f}")
    payment = payout(10, 3.5, 8.5)
    print(f"Method 2 - Payment: {payment:.2f}")
    rate3 = mr.rate_per_litre(4.2, 8.7)
    print(f"Method 3 - Rate: {rate3:.2f}")


if __name__ == "__main__":
    main()
```

6. Expected Output

```
python milkrate.py
  10 L fat 3.5 snf 8.5    rate 30.00    pay 300.0
 8.5 L fat 4.2 snf 8.7    rate 33.40    pay 283.9
  12 L fat 3.1 snf 8.3    rate 27.80    pay 333.6

python test_milkrate.py
Method 1 - Rate: 30.00
Method 2 - Payment: 300.00
Method 3 - Rate: 33.40
```

7. Test Table

Syntax	Explanation	Example
import milkrate	Imports the module namespace	milkrate.payout(10, 3.5, 8.5)
from milkrate import payout	Imports one function directly	payout(10, 3.5, 8.5)
import milkrate as mr	Uses an alias for the module	mr.rate_per_litre(4.2, 8.7)

8. Purpose of if __name__ == '__main__'

When milkrate.py is run directly, __name__ is '__main__' and the example table prints. When it is imported, __name__ is 'milkrate', so the table does not run, but the functions remain available.

9. Standard modules and sys.path

math provides mathematical functions; decimal supports exact decimal arithmetic and explicit rounding. sys.path lists the locations Python searches for imports (script directory, library paths). Imported modules are cached in sys.modules.

10. Alternative concept

Copying the functions into every script means each rate change must be repeated and risks inconsistency; a shared module is updated once and reused.

11. Complexity Analysis

A rate or payment calculation is O(1); processing n entries is O(n). The first import has a loading cost; later imports reuse the cached module. Module definitions are O(1) space relative to entry count; stored entries need O(n).

12. Conclusion

The module separates the rate card and logic from the script that uses it. The direct-run guard prevents demo output on import, and the test script shows the functions are reusable.

TASK 8 REPORT

Plan the Algorithm Before Coding | 4 Marks

1. Aim and Objective

Plan the ten-day payment algorithm before coding: write pseudocode, illustrate entry validation with a flowchart, hand-trace three entries and compare storage strategies.

2. Algorithm / Approach

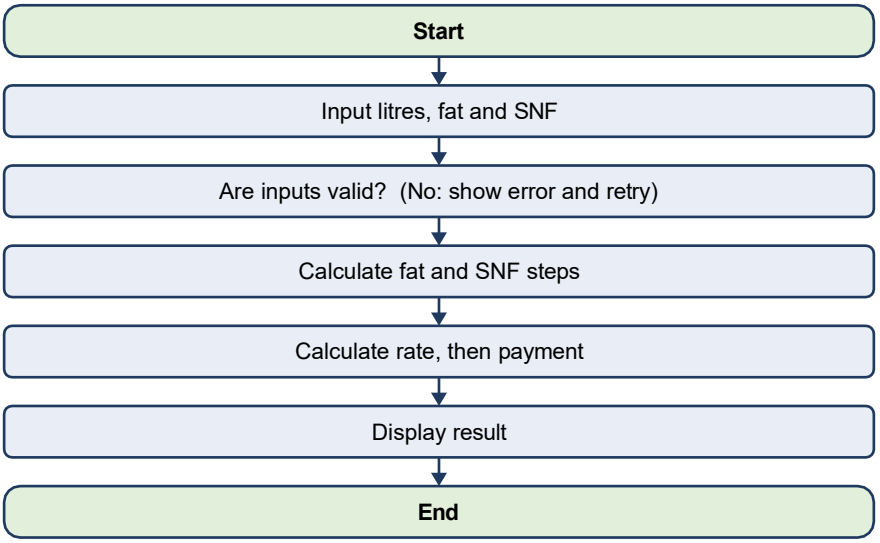
1. Understand: identify litres, fat, SNF, the rate formula and the required payment and total.
2. Plan: choose validation, rate, payment and total strategy.
3. Write: express the algorithm in pseudocode before coding.
4. Test: use normal examples, invalid inputs and a three-entry hand trace.

3. Pseudocode

```
ALGORITHM Ten_Day_Milk_Payment
  SET BASE_RATE = 30.00, FAT_STEP_VALUE = 0.40, SNF_STEP_VALUE = 0.30
  SET total_payment =
  0.00 FOR day = 1 TO
  10
    INPUT litres, fat, snf
    WHILE any input is
    invalid
      DISPLAY "Invalid
      input" INPUT litres,
      fat, snf
    END WHILE
    SET fat_steps = ROUND((fat -
    3.5) * 10) SET snf_steps =
    ROUND((snf - 8.5) * 10)
    SET rate = BASE_RATE + fat_steps*FAT_STEP_VALUE +
    snf_steps*SNF_STEP_VALUE ROUND rate TO 2 decimal places
    SET payment = ROUND(litres *
    rate, 2) DISPLAY rate, payment
    SET total_payment = total_payment +
  payment END FOR
  DISPLAY
total_payment END
ALGORITHM
```

4. UML Diagram

Activity / program flow:



5. Hand Trace of Three Entries

Day	Litres	Fat	SNF	Fat St	SNF St	Rate	Payment	Running total
1	10.0	3.5	8.5	0	0	30.00	300.00	300.00
2	8.5	4.2	8.7	7	2	33.40	283.90	583.90
3	12.0	3.1	8.3	-4	-2	27.80	333.60	917.50

Day 2: fat steps = round((4.2 - 3.5) x 10) = 7; SNF steps = round((8.7 - 8.5) x 10) = 2; rate = 30.00 + 7 x 0.40 + 2 x 0.30 = 33.40;
payment = 8.5 x 33.40 = 283.90.

6. Test Table

Test	Input	Expected rate	Expected payment	Purpose
1	10 L, fat 3.5, SNF 8.5	Rs 30.00	Rs 300.00	Reference values
2	8.5 L, fat 4.2, SNF 8.7	Rs 33.40	Rs 283.90	Above reference
3	12 L, fat 3.1, SNF 8.3	Rs 27.80	Rs 333.60	Below reference
4	Invalid text input	None	None	Validation rejects and retries

7. Alternative: storage strategy

Storing all entries and summing at the end is O(n) time and O(n) space and supports history, audit and recalculation. A running total while reading is O(n) time and O(1) extra space and suffices when only the total is needed. Validation should reject non-numeric or non-finite values and apply any permitted ranges given in the assignment.

8. Complexity Analysis

Both approaches take linear time O(n). Storing entries costs O(n) space; a running total needs only O(1) additional space.

9. Conclusion

Planning makes the calculation steps explicit before coding. The pseudocode, flowchart and hand trace use the Task 7 rate card.

TASK 9 REPORT

Implement and Test the Calculator | 4 Marks

1. Aim and Objective

Implement a milk-payment calculator, test normal, boundary and invalid inputs, and verify five sample entries by hand against program output.

2. Algorithm / Approach

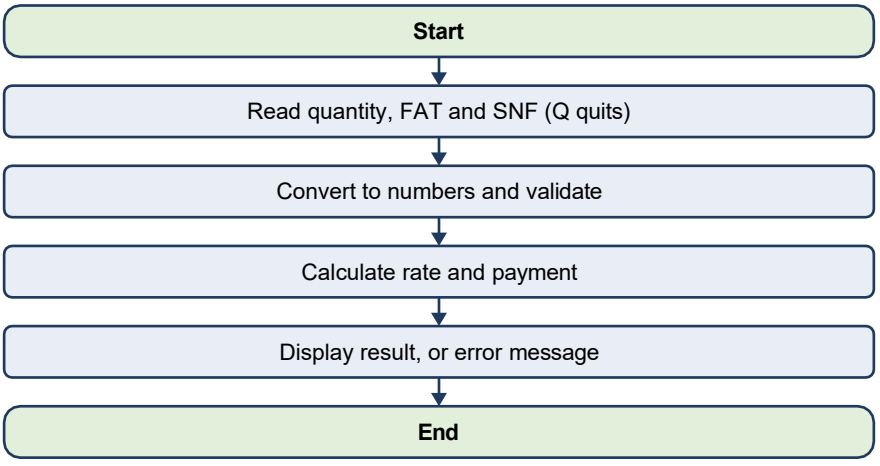
1. Set base rate, standard fat/SNF and the permitted fat range.
2. Read quantity, fat and SNF until the user enters Q.
3. Reject negative quantity, out-of-range fat or negative SNF.
4. Calculate and display rate and payment.

3. Pseudocode

```
START
  Set base rate, standard FAT/SNF and permitted FAT
  range Repeat until user enters Q:
    Read quantity, FAT and SNF; convert to numbers
    Reject negative quantity, out-of-range FAT or
    negative SNF Otherwise calculate rate and payment;
    display them
END
```

4. UML Diagram

Activity / program flow:



Class / structure diagram:

```
main()  -> reads input and handles errors
 +-- calculate_payment(quantity, fat, snf)
       +-- calculate_rate(fat, snf)
```

5. Source Code

```
# task9_calculator.py
BASE_RATE, STANDARD_FAT, STANDARD_SNF = 30.00, 3.5, 8.5
FAT_STEP_VALUE, SNF_STEP_VALUE = 0.40, 0.30
MIN_FAT, MAX_FAT = 2.0, 6.0


def calculate_rate(fat, snf):
    fs = round((fat -
    STANDARD_FAT) * 10) ss =
    round((snf -
    STANDARD_SNF) * 10)
    return round(BASE_RATE + fs * FAT_STEP_VALUE + ss * SNF_STEP_VALUE, 2)


def calculate_payment(quantity, fat,
    snf): if quantity < 0:
        raise ValueError("Milk quantity cannot be
    negative.") if not MIN_FAT <= fat <=
    MAX_FAT:
        raise ValueError(f"FAT must be between {MIN_FAT}%
    and {MAX_FAT}%.") if snf < 0:
        raise ValueError("SNF cannot be
    negative.") rate = calculate_rate(fat, snf)
    return rate, round(quantity * rate, 2)


def main():
    print("MILK PAYMENT CALCULATOR")
    while True:
        try:
            text = input("Quantity in litres (or Q to quit):
            ").strip() if text.lower() == "q":
                break
            quantity = float(text)
            fat = float(input("FAT
            percentage: ")) snf =
            float(input("SNF percentage: "))
            rate, payment = calculate_payment(quantity,
            fat, snf) print(f"Rate per litre: Rs. {rate:.2f}")
            print(f"Total payment: Rs. {payment:.2f}")
        except ValueError as error:
            print(f"Invalid input: {error}")


if _name_== "_main_
    ": main()
```

6. Sample Input

8.5
4.2
8.7
-5
3.5
8.5

7. Expected Output

Rate per litre: Rs. 33.40
Total payment: Rs.
283.90
Invalid input: Milk quantity cannot be negative.

8. Test Table

Case	Input	Expected result	Actual result
1	10 L; FAT 3.5; SNF 8.5	Rs 300.00	Fill after running
2	8.5 L; FAT 4.2; SNF 8.7	Rs 283.90	Fill after running
3	10 L; FAT 3.0; SNF 8.0	Rs 265.00	Fill after running
4	Negative quantity	Reject	Fill after running
5	FAT below minimum	Reject	Fill after running
6	FAT above maximum	Reject	Fill after running
7	Text instead of quantity	Reject with message	Fill after running
8	FAT exactly 2.0 and 6.0	Accept both	Fill after running

Five-entry hand calculation

Entry 1: 10.0 L, 3.5/8.5, rate 30.00, payment 300.00. Entry 2: 8.5 L, 4.2/8.7, rate 33.40, payment 283.90.
Entry 3: 10.0 L, 3.0/8.0, steps -5/-5, rate 30.00 - 2.00 - 1.50 = 26.50, payment 265.00. Entry 4: 5.0 L, 3.5/8.5,
rate 30.00, payment 150.00. Entry 5: 6.0 L, 4.0/8.6, rate 32.30, payment 193.80. Total = 300.00 + 283.90 +
265.00 + 150.00 + 193.80 = Rs 1,192.70.

9. Correction

The earlier draft gave entry 3 as Rs 27.50 / Rs 275.00 and the total as Rs 1,202.70. The correct values are Rs 26.50 / Rs 265.00 and Rs 1,192.70.

10. Alternative concept

Manual testing needs every case retyped after each change. Automated assert-based tests rerun fixed cases consistently.

11. Complexity Analysis

Each calculator entry is O(1) time and O(1) extra space; testing n cases takes O(n) time.

12. Limitations and Notes

- Confirm the official rate formula and allowed fat limits (the code assumes 2.0% to 6.0%).
- Record actual results only after running the program; do not invent them.

13. Conclusion

The calculator validates inputs, computes payments and supports a hand-checkable five-entry total.

TASK 10 REPORT

Compare Expectations with Execution | 4 Marks

1. Aim and Objective

Compare float and Decimal totals for 300, 6,000 and 150,000 payments; measure runtime with perf_counter(); compare rounding each payment with rounding only the total; and analyse a dictionary rate table.

2. Algorithm / Approach

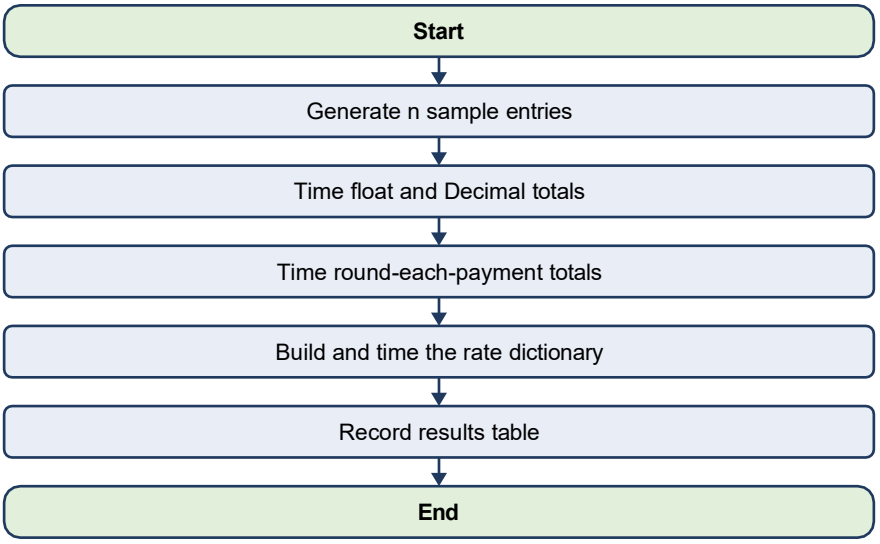
1. Generate n sample entries (litres, fat, SNF).
2. Compute the float total and the Decimal total, timing each.
3. Compute totals that round every payment to two decimals.
4. Build a dictionary of rates for distinct (fat, SNF) pairs and time the lookups.
5. Record measured results from your own run.

3. Pseudocode

```
START
  FOR n IN (300, 6000, 150000):
    data = entries(n)
    time float total ; time Decimal total
    time round-each totals (float and
    Decimal) print totals, difference and
    timings
  Build rate dictionary for distinct (fat, snf) pairs; time lookup
total END
```

4. UML Diagram

Activity / program flow:



5. Source Code

```
from decimal import Decimal,
ROUND_HALF_UP from time
import perf_counter

D01, D10 = Decimal("0.01"), Decimal("10")


def money(x):
    return x.quantize(D01, rounding=ROUND_HALF_UP)


def float_rate(fat, snf):
    return 30.0 + round((fat - 3.5) * 10) * 0.40 + round((snf - 8.5) * 10) * 0.30


def steps(value, ref):
    """Exact step count from Decimal values (rounded to an integer)."""
    return int(((value - Decimal(ref)) * D10).to_integral_value(ROUND_HALF_UP))


def decimal_rate(fat, snf):
    return (Decimal("30.00") + Decimal(steps(fat, "3.5")) * Decimal("0.40")
            + Decimal(steps(snf, "8.5")) * Decimal("0.30"))


def entries(n):
    return [(5.0 + (i % 20) * 0.25, round(3.0 + (i % 11) * 0.1, 1),
             round(8.0 + (i % 7) * 0.1, 1)) for i in range(n)]


def dec_pay(q, f, s):
    return Decimal(str(q)) * decimal_rate(Decimal(str(f)), Decimal(str(s)))


def compare(data):
    t = perf_counter()
    ft = sum(q * float_rate(f, s) for q, f, s in
    data) ftime = perf_counter() - t

    t = perf_counter()
    dt = sum((dec_pay(q, f, s) for q, f, s in data),
    Decimal("0.00")) dtime = perf_counter() - t

    t = perf_counter()
    float_each = sum(round(q * float_rate(f, s) + 1e-9, 2) for q, f, s in data)
    fe_time = perf_counter() - t

    t = perf_counter()
```

```
    dec_each = sum((money(dec_pay(q, f, s)) for q, f, s in data),
    Decimal("0.00")) de_time = perf_counter() - t

    print(f"Payments: {len(data)}")
    print(f"Float total (rounded at end): Rs. {round(ft + 1e-9,
    2):.2f}") print(f"Decimal total (rounded at end): Rs.
    {money(dt):.2f}") print(f"Difference: Rs. {round(ft -
    float(dt), 2):.2f}") print(f"Float / Decimal time: {ftime:.6f}
    / {dtime:.6f} s")
    print(f"Round-each float / Decimal: {float_each:.2f} /
    {dec_each:.2f}") print(f"Round-each time: {fe_time:.6f} /
    {de_time:.6f} s")


def
    rate_table(data)
    : t =
    perf_counter()
    table = {(f, s): float_rate(f, s) for _, f, s in data}
    build = perf_counter() - t
    t = perf_counter()
    total = sum(q * table[(f, s)] for q, f, s in
    data) lookup = perf_counter() - t
    print(f"Distinct pairs k={len(table)}; build={build:.6f}s;
    lookup={lookup:.6f}s") print(f"Lookup total: Rs. {total:.2f}; extra
    table space O(k)")


if __name__ == "__main__":
    for n in (300, 6000, 150000):
        compare(entries(n))
    rate_table(entries(150000))
```

6. How to Run

Run: python task10_comparison.py

Copy your computer's measured totals and times into the results table below.

7. Expected behaviour

Float uses binary floating point and may carry small representation errors; Decimal represents base-10 values predictably. Rounding each payment can give a different total from rounding only the grand total. Timings depend on the computer, Python version and workload.

8. Rounding analysis

Rounding each payment matches the individual amounts when farmers are paid separately. Rounding only the grand total can differ because fractions of paise are combined first. Follow the rounding rule required by the assignment or co-operative.

9. Alternative: rate dictionary

A dictionary stores each distinct (fat, SNF) pair and its rate. Lookup is O(1) average; building is O(k) time and O(k) space for k distinct pairs. It helps when pairs repeat often; if nearly every pair is unique, the memory overhead may not be worthwhile.

10. Expected versus observed

After running, state whether the totals agree, report the exact differences and timings, and explain any rounding difference. Do not claim timings without running the program.

11. Complexity Analysis

Both methods take O(N) time for N payments; Decimal has a larger constant overhead. The dataset needs O(N) space; the optional dictionary adds O(k).

12. Limitations and Notes

- Code fix: the earlier draft truncated Decimal step counts with int(), which can give wrong steps when floats such as 3.3000000000000003 are converted by str(). The code above rounds the entries and the step counts.
- Final evidence must contain measured output from your own run.

13. Conclusion

This task analyses numerical accuracy, rounding policy and rate-table lookup; results must come from your own measured output.

TASK 11 REPORT

Recommend and Defend the Final Design | 4 Marks

1. Aim and Objective

Recommend a final design for co-operative milk-payment calculations, supported by the earlier experiments, and justify the data types, rounding, rate revision, limitations and scalability.

2. Algorithm / Approach

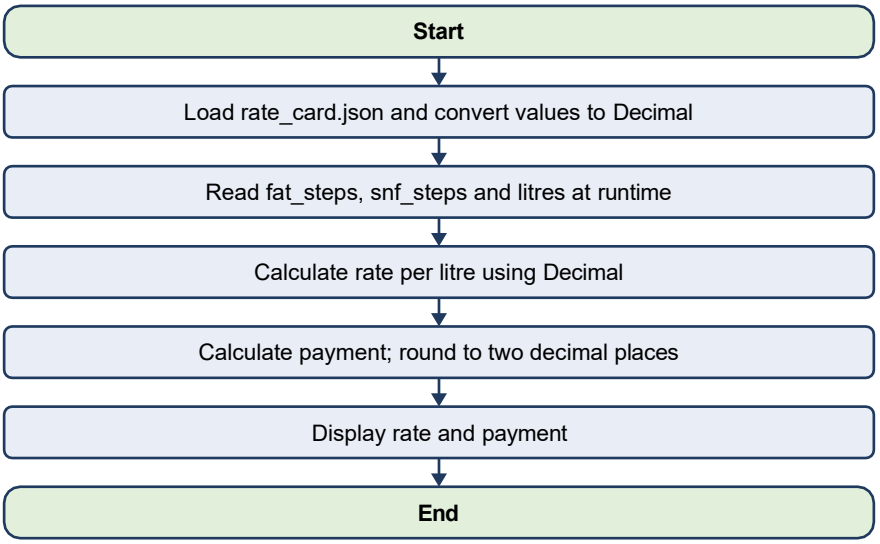
1. Use Decimal values created from decimal strings for money and rates.
2. Round payable amounts with quantize(Decimal('0.01')) using an approved rounding mode.
3. Keep the rate card in a separate JSON file so authorised revisions need no code edit.
4. Use int for discrete step counts.

3. Pseudocode

```
START
  Load rate_card.json and convert values to
  Decimal Read fat_steps, snf_steps and litres
  rate = base + fat_steps * fat_rate + snf_steps *
  snf_rate payment = quantize(litres * rate, 0.01)
  Display rate and
payment END
```

4. UML Diagram

Activity / program flow:



5. Source Code

```
from decimal import
Decimal import json
from pathlib import Path

RATE_CARD_FILE = Path("rate_card.json")


def load_rate_card():
    """Load the co-operative rate card from JSON."""
    with RATE_CARD_FILE.open("r",
        encoding="utf-8") as file: data = json.load(file)
    return {
        "base": Decimal(str(data["base"])),
        "fat": Decimal(str(data["fat"])),
        "snf": Decimal(str(data["snf"])),
    }


def calculate_rate(rate_card, fat_steps,
    snf_steps): return (rate_card["base"]
            + Decimal(fat_steps) * rate_card["fat"]
            + Decimal(snf_steps) * rate_card["snf"])


def calculate_payment(litres,
    rate_per_litre): litres =
    Decimal(str(litres))
    rate_per_litre = Decimal(str(rate_per_litre))
    return (litres * rate_per_litre).quantize(Decimal("0.01"))


if __name__ == "__main__":
    rate_card =
    load_rate_card()
    print("Rate card loaded:", rate_card)
    fat_steps = int(input("Enter fat steps:
    ")) snf_steps = int(input("Enter SNF
    steps: "))
    litres = input("Enter milk quantity in litres: ")
    rate = calculate_rate(rate_card, fat_steps,
    snf_steps) amount = calculate_payment(litres,
    rate)
    print(f"Rate per litre: Rs {rate:.2f}")
    print(f"Payment amount: Rs
    {amount:.2f}")

# rate_card.json
# { "base": "30.00", "fat": "0.40", "snf": "0.30" }
```

6. Sample Input

fat_steps = 7
snf_steps = 7
litres = 10

7. Expected Output

Rate card loaded: {'base': Decimal('30.00'), 'fat': Decimal('0.40'), 'snf': Decimal('0.30')} Rate per litre: Rs 34.90
Payment amount: Rs 349.00

Input prompts omitted. Values are illustrative, not official Aavin rates.

8. Alternative Design Comparison

Approach	Complexity	Evaluation
Embedded constants	No file read; revisions need a code edit and redeployment	Simple for a fixed demo
External JSON	O(t) load time and memory for t values	Recommended for controlled rate revisions

9. Evidence from earlier tasks

Task 2: 0.1 + 0.2 gives 0.30000000000000004 and equality with 0.3 is False, while Decimal strings compare exactly; 1,000 x Rs 33.40 plus three x Rs 0.10 totals Rs 33,400.30 with Decimal but 33400.30000000061 with float. Task 4: (4.2 - 3.5) / 0.1 is 7.000000000000002; round() yields 7. Task 1 uses a manually entered rate and does not establish an official formula.

10. Data types and rounding

Use Decimal('30.00'), not Decimal(30.00), for money and rates. Use int for discrete step counts. Round at the payment boundary with quantize(Decimal('0.01')) and an explicitly approved rounding mode.

11. Rate revision procedure

1) An authorised officer updates rate_card.json from approved rates. 2) Validate keys, values, ranges and effective date. 3) Keep dated versions and approval details for audit. 4) Test known and boundary cases. 5) Reload the JSON file; source code need not change. 6) Record the rate-card version used for each payment.

12. Limitations and scalability

The demo lacks authentication, database persistence, audit logs, backups, concurrent updates and official rate logic. District-union deployment needs a database, controlled rate changes, version history, logging, access control, backups and concurrency testing.

13. Complexity Analysis

For t rate values, JSON loading is O(t) time and memory. One record calculation is O(1); processing n records is O(n) time and storing them is O(n) space. JSON's main benefit is maintainability, not faster arithmetic.

14. Conclusion

Recommend Decimal for money, a documented two-decimal rounding rule and a validated external JSON rate card. Production use also needs validation, auditability, security and database support.

TASK 12 REPORT

Milk Payment System - Seasonal Rates and Bonus

1. Aim and Objective

Re-evaluate the original design after the analyser supplies fat and SNF to two decimal places and the union announces seasonal rates and a ten-day supply bonus.

2. Algorithm / Approach

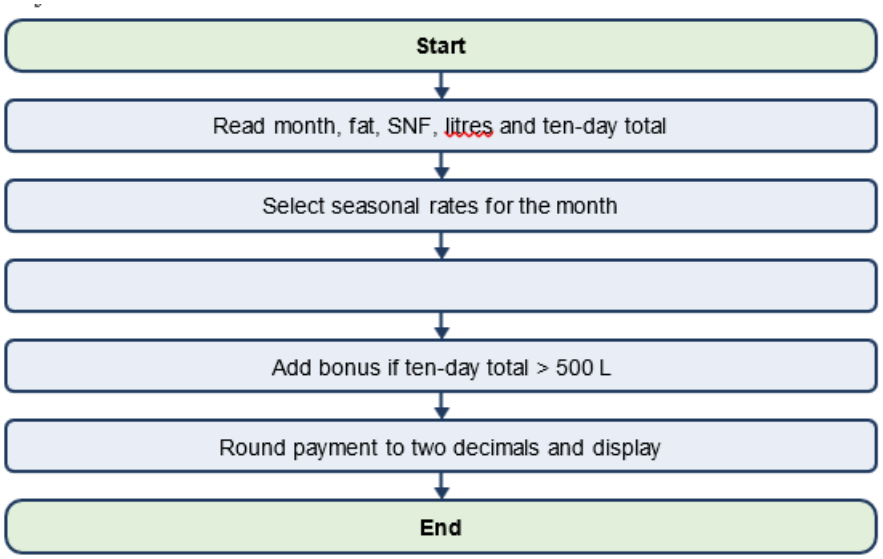
1. Decision: MODIFY the original design rather than redesign it. Keep Decimal money arithmetic and the separate rate-card configuration.
2. Update the April-June base rate to Rs 32.00 and the fat-step rate to Rs 0.45; add a seasonal-rate selection function.
3. Accept two-decimal readings using Decimal from strings; never convert readings to int.
4. Add Rs 0.50 per litre only when the ten-day total is strictly greater than 500 litres (exactly 500 does not qualify).
5. Round the final payment to two decimal places.

3. Pseudocode

```
START
  Read month, fat, snf, today's litres, ten-day
  total (base, fat_rate, snf_rate) =
  get_rates(month) fat_steps = (fat -
  reference_fat) / step_size snf_steps = (snf -
  reference_snf) / step_size
  rate = base + fat_steps*fat_rate +
  snf_steps*snf_rate bonus = 0.50 IF
  ten_day_total > 500 ELSE 0.00
  payment = quantize((rate + bonus) * litres,
  0.01) Display steps, rates, bonus and
  payment
END
```

4. UML Diagram

Activity / program flow:



Source Code

```
from decimal import Decimal, ROUND_HALF_UP


def get_rates(month):
    # SNF rate is illustrative: the changed requirement does not
    specify it. if 4 <= month <= 6:
        return      Decimal("32.00"),      Decimal("0.45"),
    Decimal("0.30") # Outside April-June, retain the
    original illustrative rates. return Decimal("30.00"),
    Decimal("0.40"), Decimal("0.30")


def calculate_payment(fat, snf, litres_today, total_10_days, month,
                      reference_fat="3.00",
                      reference_snf="8.00", step_size="0.10"):
    fat =
    Decimal(str(fat)) snf
    = Decimal(str(snf))
    litres_today =
    Decimal(str(litres_today))
    total_10_days =
    Decimal(str(total_10_days))
    reference_fat = Decimal(reference_fat)
    reference_snf = Decimal(reference_snf)
    step_size = Decimal(step_size)

    base_rate, fat_step_rate, snf_step_rate =
    get_rates(month) fat_steps = (fat - reference_fat) /
    step_size
```

```
    snf_steps = (snf - reference_snf) / step_size
    rate = base_rate + fat_steps * fat_step_rate + snf_steps * snf_step_rate

    bonus_rate = Decimal("0.50") if total_10_days > Decimal("500") else
    Decimal("0.00") final_rate = rate + bonus_rate
    payment = (litres_today * final_rate).quantize(
        Decimal("0.01"),
        rounding=ROUND_HALF_UP)
    return fat_steps, snf_steps, rate, bonus_rate, final_rate, payment


if __name__ == "__main__":
    month = int(input("Enter month number
    (1-12): ")) fat = input("Enter fat reading
    (e.g. 4.25): ") snf = input("Enter SNF
    reading (e.g. 8.50): ")
    litres = input("Enter today's milk quantity in litres: ")
    total_10_days = input("Enter total supply in 10 days:
    ")
    result = calculate_payment(fat, snf, litres, total_10_days,
    month) fat_steps, snf_steps, rate, bonus, final_rate, payment
    = result print(f"Fat steps: {fat_steps}")
    print(f"SNF steps: {snf_steps}")
    print(f"Rate before bonus: Rs
    {rate:.2f}") print(f"Bonus per litre:
    Rs {bonus:.2f}")
    print(f"Final rate per litre: Rs {final_rate:.2f}")
    print(f"Payment for today's delivery: Rs
    {payment:.2f}")
```

5. Sample Input

```
Enter month number (1-
12): 5 Enter fat reading:
4.25 Enter SNF
reading: 8.50
Enter today's milk quantity in litres:
20 Enter total supply in 10 days: 520
```

6. Expected Output

```
Fat steps: 12.50
SNF steps: 5.00
Rate before bonus: Rs 39.12 (exactly
39.125) Bonus per litre: Rs 0.50
Final rate per litre: Rs 39.62 (exactly
39.625) Payment for today's delivery: Rs
792.50
```

Hand-calculated with the illustrative values: 32.00 + 12.5 x 0.45 + 5 x 0.30 = 39.125; plus bonus 0.50 = 39.625; x 20 L = Rs

792.50. Run the program to confirm. The displayed per-litre rates are rounded for display only; the payment uses the exact rate.

7. Test Table

Test	Expected result
April-June month (4 to 6)	Rs 32.00 base, Rs 0.45 fat step
March and July	Original illustrative rates
Ten-day total exactly 500 L	No bonus
Ten-day total 500.01 L	Bonus Rs 0.50 per litre
Fat 4.25 with reference 3.00, step 0.10	12.5 steps (not truncated)

8. Open points to confirm

The requirement does not give the SNF-step rate, the reference fat/SNF readings, the step size, the policy for fractional steps, or whether the bonus applies to all litres in the ten days or only the current delivery. The code uses illustrative values for these, which are not official union rates. Reference values (fat 3.00, SNF 8.00) differ from the earlier tasks (3.5 and 8.5) and must be confirmed. Test the March/April and June/July season boundaries.

9. Complexity Analysis

Per record O(1) time and space; n records O(n).

10. Limitations and Notes

- Rates and reference/step values not stated in the changed requirement are illustrative assumptions.

11. Conclusion

The Decimal-based design remains suitable but must be modified to support seasonal configuration, decimal analyser readings and the ten-day bonus. Confirm unspecified business rules before deployment.